Exercise: The COPY Command

Learn how to load transformed data into a PostgreSQL database using the COPY command. This lesson guides you through creating a table and function to automate loading CSV data in an ETL pipeline, including truncating the table before each load and verifying the insertion.

In this exercise, we'll practice using the `COPY` command to load data into a PostgreSQL database. Our task is to create a table named `cars` and a function that loads a CSV file into it.

This function will be a part of the load step in an ETL pipeline that uses full loads. Therefore, we need to truncate the table each time we load new data.

Create a table

The CSV file called `cars.csv` contains a million records, and has the following format:

main.txt C++

```
ProductID,Company,CarName,Price,Quantity
1,Toyota,Camry,89915.37,7
2,Tesla,Model S,34397.88,6
3,Volvo,XC60,88356.36,9
4,Ferrari,SF90488,19463.69,7
5,Mazda,CX-5,69653.82,4
6,Toyota,Camry,41087.01,1
7,Tesla,Model S,71948.6,10
8,Jeep,Wrangler,73004.64,2
9,Toyota,Camry,52439.4,7
```

Ask

Sample data: cars.csv

Our first step is to create a table to store this data. Create a table with matching attributes, names, and data types to load the data in later steps.

main.sql PostgreSQL

```
CREATE TABLE <<InsertHere>>(
  ProductID int not null primary key,
  <<InsertHere>>,
  <<InsertHere>>,
  <<InsertHere>>,
  <<InsertHere>>
)
```

Run

Creating a table

Finding the secured file location

Before loading the data, we should check where the secured file location for PostgreSQL is located, and make sure that the file `cars.csv` is stored there.

By default, PostgreSQL doesn't allow imports and exports from unauthorized directories. PostgreSQL keeps the location of the secured file location in the variable `data_directory`. We can use the `SHOW data_directory` command to check its contents.

Ask

For this exercise, the location of the secured file location is: `/usercode/tmp`

Create a function

In PostgreSQL, we can store scripts that we wrote for later use. Since this process will be used as the loading step in an ETL pipeline, we'll use this script in the future. Therefore, it would be a good practice to store it as a function.

Create a function that truncates the table `cars` and then uses the `COPY` command to load the data. Finally, return the number of rows inserted as output.

main.sql PostgreSQL

```
-- Creating a function called load_data
CREATE OR REPLACE function load_data()
RETURNS integer
LANGUAGE plpgsql
AS $$

-- Declaring a variable to store the number of rows inserted
declare
  number_of_rows int;

-- Begin transaction
BEGIN
```

Run

Stored procedure

Finally, query the table `cars` to make sure everything has worked.

main.sql PostgreSQL

Ask

```
select *
from cars
limit 20;
```

Run

Create a table solution

main.sql PostgreSQL

```
CREATE TABLE cars(
  ProductID int not null primary key,
  Company varchar(25) not null,
  CarName varchar(25) not null,
  Price float not null,
  Quantity int not null
)
```

Run

Creating a table

Solution code: Create a function

Here's the solution code:

Ask

main.sql PostgreSQL

```
-- Creating a function called load_data
```

```
CREATE OR REPLACE function load_data()
RETURNS integer
LANGUAGE plpgsql
AS $$

-- Declaring a variable to store the number of rows inserted
declare
  number_of_rows int;

-- Begin transaction
BEGIN
```

Run

Solution code: Stored procedure

- **Line 2**: We create a function called `load_data()`.
- **Line 3**: We set the data type of the return value.
- **Line 4**: We specify that `pgplsql` is the language to be used
- **Lines 8–9**: We declare a variable `number_of_rows` to store the number of inserted rows.
- **Line 15**: We truncate the table `cars`.
- **Lines 18–21**: We use the `COPY` command to load data from the secured file located at: `/usercode/tmp/cars.csv`.
- **Lines 24–26**: We count the number of rows inserted and store it in the variable `number_of_rows`.
- **Line 29**: We print a message accordingly.
- **Line 31**: We return `number_of_rows`.
- **Line 35**: We call the `load_data()` function.

Mark As Completed

Ask